

JAVA FOUNDATIONS

GENERICICS

Dr Vincent Chung
shchung@nus.edu.sg

Example: `<T>` `<?>`
Placeholder T for Type
E for element

Scenario

Look at the following code

```
static double sum(int[] arr) {  
    double total = 0;  
    for (int e: arr) {  
        total += e;  
    }  
    return total;  
}  
  
static double sum(float[] arr) {  
    double total = 0;  
    for (float e: arr) {  
        total += e;  
    }  
    return total;  
}  
  
static double sum(double[] arr) {  
    double total = 0;  
    for (double e: arr) {  
        total += e;  
    }  
    return total;  
}
```

```
public static void main(  
    String[] args){  
    int[] intArr = {1, 2, 3};  
    System.out.println("Sum is " +  
        sum(intArr));  
  
    float[] floatArr =  
        {1.1f, 2.2f, 3.3f};  
    System.out.printf("Sum is " +  
        sum(floatArr));  
  
    double[] doubleArr =  
        {1.1, 2.2, 3.3};  
    System.out.printf("Sum is " +  
        sum(doubleArr));  
}
```



3 overloaded methods

sum() almost do the **same** thing. Only the **types** of array are **different**.

How can we avoid those **duplicate** codes?

A Polymorphism Attempt

```
static double sum(
    Object[] arr)
{
    double total = 0;
    for (Object e: arr)
    {
        total += e;
    }
    return total;
}
```

```
public static void main(String[] args) {
    Object[] intArr = {1, 2, 3};
    System.out.println("Sum is "
        + sum(intArr));

    Object[] floatArr = {1.1f, 2.2f, 3.3f};
    System.out.println("Sum is "
        + sum(floatArr));

    Object[] doubleArr = {1.1, 2.2, 3.3};
    System.out.println("Sum is "
        + sum(doubleArr));
}
```



Does it work?

Problems with the Polymorphism Attempt

1. Operator `+` is not applicable for class **Object**
2. Even when it's possible, we can **mistakenly add a non-integers** to *intArr*, which will produce **run-time bugs**



Image by [Jan](#) from [Pixabay](#)

- **What is Generics?**
- Generic Types and Generic Methods
- Bounded Generic Types
- Generic Classes

Generics:

Parameterized Types: Instead of specifying a concrete type (like `int` or `String`) initially, you use a placeholder (e.g., `T`).

Type Safety: The compiler verifies that you are using the correct data type, preventing bugs before the code runs.

Reusability: A single, generic algorithm or data structure can handle multiple types.

What is Generics?

Shared by, typical of, or **relating to** a whole **group of similar** things, **rather than** to any **particular** thing



Photo by [Robert Anasch](#) on [Unsplash](#)

The Cambridge Advanced Learner's Dictionary

Java Generics

The capability to define **classes** or **methods** with **generic types**



Photo by [Raul Cachó Oses](#) on [Unsplash](#)

- Enable the **creation of classes** and **methods** that work with different data types while providing **compile-time type safety** and eliminating the need for explicit type casting.
- Generics provide a mechanism for **parameterizing types**, a single class or method can be written using a placeholder for a data type. The actual data type is then specified by the user **during instantiation**, using **angle brackets (<>)**.

Generic Types and Methods

A **generic type** 1. represents **data type** that 2. **can be used later**

```
static ① <T> void printArr(② T[] arr) {
    for (int i=0; i<arr.length; i++)
    {
        ② T t = arr[i];
        System.out.print(t + " ");
    }
    System.out.println();
}
```

① Declare T as a generic type. Anything can be used, but T (type), E (element), K (key) and V (value) are commonly used

② T can be used as a parameter type

② T can also be used as a variable type

Modifier	Declared generic type	Return type	Name	Parameter that uses generic type
static	<T>	void	printArr	(T[] arr)

Generic Types and Methods

The **data type** will be **specified** at **run time**

```
static <T> void printArr(T[] arr)
{
    for (int i=0; i<arr.length; i++)
    {
        T t = arr[i];
        System.out.print(t + " ");
    }
    System.out.println();
}
```

```
public static void main(String[] args)
{
    Integer[] intArr = { 1, 2, 3 };
    printArr(a intArr);

    Double[] doubleArr =
        { 1.1, 2.2, 3.3 };
    printArr(b doubleArr);

    String[] stringArr =
        { "aa", "bb", "cc" };
    printArr(c stringArr);
}
```

a. The generic type *T* is specified as *Integer* at run time

b. The generic type *T* is specified as *Double* at run time

c. The generic type *T* is specified as *String* at run time

Generic Types and Methods

In this case, *behind the scene*, compiler will **create 3 different versions**. For example,

1

```
static void printArr(Integer[] arr) {  
    for (int i=0; i<arr.length; i++) {  
        Integer t = arr[i];  
        System.out.print(t + " ");  
    }  
    System.out.println();  
}
```

3

```
static void printArr(String[] arr) {  
    for (int i=0; i<arr.length; i++) {  
        String t = arr[i];  
        System.out.print(t + " ");  
    }  
    System.out.println();  
}
```

2

```
static void printArr(Double[] arr) {  
    for (int i=0; i<arr.length; i++) {  
        Double t = arr[i];  
        System.out.print(t + " ");  
    }  
    System.out.println();  
}
```

Outline

- What is Generics?
- **Generic Types and Generic Methods**
- Bounded Generic Types
- Generic Classes

Review – Variables

Variables 1. represent **values** that 2. **can be used later** in the program. 3. Their **values** may be **changed**

```
public static void main(String[] args) {
    double 1radius, area;
2radius = 1.0;
    area = 2radius * radius * 3.14;
    System.out.println("Area " + area +
        " for radius " + 2radius);

3radius = 2.0;
    area = 2radius * radius * 3.14;
    System.out.println("Area " + area +
        " for radius " + 2radius);
}
```

1. Declare variable *radius*

2. Then use it in several places

3. We can change its value

Generic Types and Methods

A **generic type** can also be **used** as **return type**

```
static <T> 1 T getElement(  
    T[] arr, int index) {  
    if (index <= arr.length - 1) {  
        return 2a arr[index];  
    }  
  
    return 2b null;  
}
```

1. Declare this method will return a value type *T*

2. All return statement must make sure that the return values are references objects typed *T*

```
public static void main(String[] args) {  
    String[] stringArr = { "aa", "bb", "cc" };  
  
    3 String str1 =  
        getElement(stringArr, 1);  
    String str2 =  
        getElement(stringArr, 3);  
    System.out.println("str1 is " + str1  
        + " and str2 is " + str2);  
}
```

3. The generic type return type *T* is specified as *String* at run time



How can the compiler know ***String*** is the **return type** when calling the *getElement()* method above?

A: Data type of *stringArr* is *String[]*. Because it is used as the generic parameter, compiler can infer that *T* is *String*

Practice

Write a method that, **given an array and an element**, returns **T/F** if the **array contains the element**

The method should **support different types of arrays**.

E.g., array of *Integers*, array of *Doubles*, array of *Strings*...



Image by [Arek Socha](#) from [Pixabay](#)

Practice Solution

Use generic methods to support different types of arrays

```
static <T> boolean isContained(T[] arr, T element) {
    for (T curElement : arr) {
        if (curElement.equals(element)) {
            return true;
        }
    }
    return false;
}
```



```
public static void main(String[] args) {
    Integer[] intArr = {1, 2, 3};
    System.out.println(isContained(intArr, 4));

    String[] stringArr = {"aa", "bb", "cc"};
    System.out.println(isContained(stringArr, "bb"));
}
```

false
true

Generic types can only represent reference types, not primitive types

Quiz

Write a method that returns the **maximum** among **3 items** of the **same type**, e.g., 3 *Integers*, or 3 *Doubles*, or 3 *Strings*, etc.

```
static int max (  
    int e1, int e2, int e3) {  
    int max = e1;  
  
    if (e2 > max) {  
        max = e2;  
    }  
  
    if (e3 > max) {  
        max = e3;  
    }  
  
    return max;  
}
```



If we **just use a generic type** as in the previous slides, we **can't compare 2 items**. How can we do that?

Outline

- What is Generics?
- Generic Types and Generic Methods
- **Bounded Generic Types**
- Generic Classes

Bounded Generic Types

A generic type can be specified as a **subtype** of another type. Such generic type is called ***bounded***

```
public static <T extends Animal>
    void makeNoiseNTimes(T myAni, int N)
{
    for (int i=0; i<N; i++) {
        myAni.makeSound();
    }
}
```

Animal
name
getName() makeSound()



Can variable *myAni* call
method *getName()*?

Bounded Generic Types

Variables of a **bounded generic type** can access the **respective methods**

```
public static <T extends Animal>
    void makeNoiseNTimes(T myAni, int N)
{
    for (int i=0; i<N; i++) {
        myAni.makeSound();
    }
}
```

Animal
name
getName() makeSound()

Question

Which statement(s) is/are allowed?

```
public static <T extends Animal>
    void makeNoiseNTimes(T myAni, int N) {
    for (int i=0; i<N; i++) {
        myAni.makeSound();
    }
}
```

Animal
name
getName() makeSound()

Baby
name
getName() makeSound()



Cat
makeSound()

- A. *makeNoiseNTimes(new Animal(), 3);*
- B. *makeNoiseNTimes(new Cat(), 2);*
- C. *makeNoiseNTimes(new Baby(), 4);*

Generic interface Comparable<T>

This interface imposes the **ordering** on the objects of each class that implements it

```
interface Comparable<T> {  
    // Compares this object with the another  
    // object for order  
    // Return negative value if this < another  
    // Return positive value if this > another  
    // Return 0 if this = another  
    int compareTo(T another);  
}
```

All the **number classes** (e.g., *Integer*, *Float*, *Double*...) and even the *String* **class** have already **implemented** interface *Comparable*

Bounded Generic Types

Besides extending a class, **subtype** also includes **implementing an interface**

```
static 1 <T extends Comparable<T>>
    T max(T e1, T e2, T e3) {
    T max = e1;
    if ( 2 e2.compareTo(max) > 0) {
        max = e2;
    }
    if (e3.compareTo(max) > 0) {
        max = e3;
    }
    return max;
}
```

1. Here, we declare a generic T to be used, just like the previous slides. Besides, we say that **only types that implements Comparable<T> interface** is acceptable.

Note that even *Comparable<T>* is an interface, in generics, we use *extends* instead of *implements* keyword

Multiple bounds are allowed with syntax *<T extends SuperClass & Interface1 & Interface2>*

```
static void main(
    String[] args) {
    System.out.println(
        max(1, 2, 3));
    System.out.println(
        max(1.1, 3.3, 2.2));
    System.out.println(
        max("cc", "aa",
            "cc"));
}
```

```
3
3.3
cc
```

2. Because *e1*, *e2*, *e3* reference instances of *T*, which implements *Comparable<T>* interface, **all methods of Comparable<T> interface** can be used, including *compareTo()*

Quiz

Write a generic method that returns the **sum** of a list of **number items** (e.g., sum of *short*, sum of *int*, sum of *float*... items). The **return type** should be *double*

Hint: all such classes extend class *Number*, which includes *doubleValue()* method that returns the value of the respective object as *double*



Image by [Gerd Altmann](#) from [Pixabay](#)

Quiz Solution

```
static <T extends Number> double sum(List<T> list) {  
    double sum = 0;  
    for (T e : list) {  
        sum += e.doubleValue();  
    }  
  
    return sum;  
}
```



In this case, do we really need to use *T* in the method body?

```
public static void main(String[] args) {  
    List<Integer> intList = new ArrayList<>();  
    intList.add(1);  
    intList.add(2);  
    intList.add(3);  
    System.out.println(sum(intList));  
  
    List<Double> doubleList = new ArrayList<>();  
    doubleList.add(1.1);  
    doubleList.add(2.2);  
    doubleList.add(3.3);  
    System.out.println(sum(doubleList));  
}
```

Quiz Solution

Because *doubleValue()* method is from class *Number*, polymorphism with *Number* has been already good enough

Alternative to *sum()*, use the simpler version *sum2()*

```
static <T extends Number> double sum(List<T> list) {
    double sum = 0;
    for (T e : list) {
        sum += e.doubleValue();
    }

    return sum;
}
```

```
static <T extends Number> double sum2(List<T> list) {
    double sum = 0;
    for (Number e : list) {
        sum += e.doubleValue();
    }

    return sum;
}
```



But now, we declare the parameterized class *T* to **use it only once**? Is there a better way?

Outline

- What is Generics?
- Generic Types and Generic Methods
- Bounded Generic types
- **Generic Classes**

Generic Classes

We can also **declare** some **generic type** and **use** it **later** in **classes**

```
class AGenericClass ① <T> {  
    private ② T data;  
  
    AGenericClass() {}  
  
    public void setData(② T newValue) {  
        data = newValue;  
    }  
    public ② T getData() {  
        return data;  
    }  
}
```

1. Declare a generic type *T*

2. Use it in various places in the class

- As **data type** of the **instance attributes**
- As **data type** of the **method arguments**
- As methods' **return type**

Using Generic Classes

We need to **specify** the **generic types** when using generic classes

```
public static void main(String[] args) {
    AGenericClass① <String> str =
        new AGenericClass<>();
    str.setData(② "This is a String");
    System.out.println(str.getData());

    AGenericClass<Integer> intg =
        new AGenericClass<Integer>();
    intg.setData(10);
    System.out.println(intg.getData());
}
```

1. Specify the generic type to be *String*. All the generic type *T* in *AGenericClass* is replaced by *String*
2. Therefore, *setData(T)* becomes *setData(String)* and we can put a string as the parameter

```
class AGenericClass <T> {
    private T data;

    AGenericClass() {}

    public void setData(
        T newValue) {
        data = newValue;
    }

    public T getData() {
        return data;
    }
}
```

Practice

Given the declaration of class *ArrayList*, its empty constructor and its method *add()*, as following

```
class ArrayList<E> {  
    ArrayList() {}  
    // Other code omitted  
    void add(int index, E element) {  
        // Method body is omitted  
    }  
    boolean add(E e) {  
        // Method body is omitted  
    }  
}
```

Write code to create an *ArrayList* object to keep a **list of names**

Then write code add the 2 following names into the beginning of the list “*Wolfgang Amadeus Mozart*” and “*Ludwig van Beethoven*”

Practice Solution

```
static void testArrayList() {  
    String name1 = "Wolfgang Amadeus Mozart";  
    String name2 = "Ludwig van Beethoven";  
  
    ArrayList<String> nameList = new ArrayList<>();  
    nameList.add(0, name1);  
    nameList.add(0, name2);  
}
```

Practice

Given the declaration of class *HashMap*, its empty constructor and its *put()* method as following

```
class HashMap<K,V> {  
    HashMap() {}  
    // Other code omitted  
    V put(K key, V value) {  
        // Method body is omitted  
    }  
}
```

Write code to create a *HashMap* object to keep a **collection of names**, where name is the key and its string length is the value

Then write code put the 2 following names into the *HashMap*:
“*Wolfgang Amadeus Mozart*” and “*Ludwig van Beethoven*”

Practice Solution

```
static void testHashMap() {  
    String name1 = "Wolfgang Amadeus Mozart";  
    String name2 = "Ludwig van Beethoven";  
  
    HashMap<String, Integer> nameToLength = new HashMap<>();  
    nameToLength.put(name1, name1.length());  
    nameToLength.put(name2, name2.length());  
}
```

Readings

- Java How to Program, 10ed Late Object version – Chapter 20, Generic Classes and Methods, *Paul Deitel and Harvey Deitel*
- Introduction to Java Programming and Data Structures, 11ed – Chapter 19, Generics, Y. *Daniel Liang*
- Class Collections [source code](#)